

Lab 4 作業說明

- 題目以及圖片可至以下網址下載

題目：http://www.imageprocessingplace.com/DIP3E/dip3e_student_projects.htm

圖片：http://www.imageprocessingplace.com/DIP3E/dip3e_book_images_downloads.htm

- 相關繳交說明請見公告區
- 題目規範 (實際題目內容請見連結)

Proj05-01: Noise Generators

需繳交的 function (命名&格式限定)：

MATLAB

```
output_s = addGaussianNoise(input_s, mu, sigma);  
output_s = addImpulseNoise(input_s, Ps, Pp);
```

Python

```
output_s = addGaussianNoise(input_s, mu, sigma)  
output_s = addImpulseNoise(input_s, Ps, Pp)
```

變數 (命名不限定)：

MATLAB

input_s & output_s: 2-D image in spatial domain, type uint8, range 0~255
mu, sigma: parameters of gaussian noise (c.f. 4/e, eq. 5-3)
Ps, Pp: parameters of salt noise and pepper noise respectively (c.f. 4/e, eq. 5-16)

Python

input_s & output_s: 2-D numpy array in spatial domain, dtype=np.uint8, range 0~255
mu, sigma: parameters of gaussian noise (c.f. 4/e, eq. 5-3)
Ps, Pp: parameters of salt noise and pepper noise respectively (c.f. 4/e, eq. 5-16)

使用的圖片：

Fig. 5.7(a)

報告：

- (1) 請重複第四版課本 Fig.5.7(a),(b); Fig.5.8(a),(b); Fig.5.10(a) (共 5 張圖)
- (2) 實作 denoise 的結果，API 可自訂 (bonus)
- (3) 任何想比較討論的內容或圖片，或者是實作心得

註 1：不可使用 `imnoise()`，可使用適當的 `random` 類函數

註 2：Gaussian noise 請直接疊加後 normalized 回 range 0~255

註 3：若 5-3 有實作 PSNR，則可於 bonus 部分進行相關比較

Proj05-02: Noise Reduction Using a Median Filter

需繳交的 function (命名&格式限定) :

MATLAB

```
output_s = myMedianFilter(input_s);
```

Python

```
output_s = myMedianFilter(input_s)
```

變數 (命名不限定) :

MATLAB

input_s & output_s: 2-D image in spatial domain, type uint8, range 0~255

Python

input_s & output_s: 2-D numpy array in spatial domain, dtype=np.uint8, range 0~255

使用的圖片 :

Fig. 5.7(a)

報告 :

- (1) 修改 Project 03-03 的程式，實作 3 x 3 median filtering
- (2) 使用 Proj05-01 的 addImpulseNoise() 對 Fig. 5.7(a) 加入 salt-and-pepper noise · $P_a = P_b = 0.2$
- (3) 對 noisy image 做 median filtering，並說明結果與 Fig. 5.10(b) 的主要差異
- (4) 任何想比較討論的內容或圖片，或者是實作心得

註 1：不可使用 `medfilt2` (MATLAB) / `scipy.ndimage.median_filter` 或 `cv2.medianBlur` (Python)

Proj05-03: Periodic Noise Reduction Using a Notch Filter

需繳交的 function (命名&格式限定) :

MATLAB

```
output_s = addSinNoise(input_s, A, u0, v0);  
[output_f, Notch] = notchFiltering(input_f, D0, u0, v0);  
psnr = computePSNR(input1_s, input2_s);
```

Python

```
output_s = addSinNoise(input_s, A, u0, v0)  
output_f, Notch = notchFiltering(input_f, D0, u0, v0)  
psnr = computePSNR(input1_s, input2_s)
```

變數 (命名不限定) :

MATLAB

input_s & output_s: 2-D image in spatial domain, type single, range 0~1
input_f & output_f: 2-D image in frequency domain (centered), type single
Notch: notch reject filter in frequency domain (two-circle version)
u0 & v0 & A: 請參考下式 · 其中 (M,N) 為 image size · A 為 amplitude of noise

$$\eta(x, y) = A \sin\left(2\pi\left(\frac{u_0 x}{M} + \frac{v_0 y}{N}\right)\right)$$

D0 & u0 & v0: 請參考第四版課本 eq. 5-34, 5-35 · 使用 ideal 版本 · D0 為半徑 · 或參考下圖的公式說明

The transfer function of an ideal notch reject filter of radius D_0 , with centers at (u_0, v_0) and, by symmetry, at $(-u_0, -v_0)$, is

$$H(u, v) = \begin{cases} 0 & \text{if } D_1(u, v) \leq D_0 \quad \text{or} \quad D_2(u, v) \leq D_0 \\ 1 & \text{otherwise} \end{cases} \quad (5.4-5)$$

where

$$D_1(u, v) = [(u - M/2 - u_0)^2 + (v - N/2 - v_0)^2]^{1/2} \quad (5.4-6)$$

and

$$D_2(u, v) = [(u - M/2 + u_0)^2 + (v - N/2 + v_0)^2]^{1/2}. \quad (5.4-7)$$

psnr: Peak Signal-to-Noise Ratio between original image and restored image

Python

input_s & output_s: 2-D numpy array in spatial domain, dtype=np.float32, range 0~1
input_f & output_f: 2-D numpy array in frequency domain (centered), dtype=np.complex64
Notch: 2-D numpy array (notch reject filter), dtype=np.float32
u0 & v0 & A: 請參考下式 · 其中 (M,N) 為 image size · A 為 amplitude of noise

$$\eta(x, y) = A \sin\left(2\pi\left(\frac{u_0 x}{M} + \frac{v_0 y}{N}\right)\right)$$

D0 & u0 & v0: 請參考第四版課本 eq. 5-34, 5-35 · 使用 ideal 版本 · D0 為半徑 · 或參考下圖的公式說明

The transfer function of an ideal notch reject filter of radius D_0 , with centers at (u_0, v_0) and, by symmetry, at $(-u_0, -v_0)$, is

$$H(u, v) = \begin{cases} 0 & \text{if } D_1(u, v) \leq D_0 \text{ or } D_2(u, v) \leq D_0 \\ 1 & \text{otherwise} \end{cases} \quad (5.4-5)$$

where

$$D_1(u, v) = [(u - M/2 - u_0)^2 + (v - N/2 - v_0)^2]^{1/2} \quad (5.4-6)$$

and

$$D_2(u, v) = [(u - M/2 + u_0)^2 + (v - N/2 + v_0)^2]^{1/2}. \quad (5.4-7)$$

psnr: Peak Signal-to-Noise Ratio between original image and restored image

使用的圖片：

Fig.5.26(a)

報告：

- (5) 請放上原圖 Fig.5.26(a) (1 張圖)
- (6) 在 spatial domain 加上 noise 後 · spatial domain 上的結果 (1 張圖)
- (7) (2) 轉到 frequency domain 上的結果 (1 張圖)
- (8) 用 notchFiltering() 造出來的 Notch filter (1 張圖)
- (9) (3) 通過 (4) 後 frequency domain 上的結果 (1 張圖)
- (10) (5) 轉回 spatial domain 上的結果 (1 張圖) (共 6 張圖)
- (11) 列出原圖 Fig.5.26(a) 和經過 (1)-(6) 步驟後結果之間的 PSNR
- (12) 任何想比較討論的內容或圖片 · 或者是實作心得

註 1：不可使用 imnoise()

註 2：u0 和 v0 請自己設定 · 最大可以代到 M/2-1 以及 N/2-1

Proj05-04: Parametric Wiener Filter

需繳交的 function (命名&格式限定) :

MATLAB

```
[output_f, H] = addMotionBlur(input_f, T, a, b);  
output_f = wienerFiltering(input_f, H, K);
```

Python

```
output_f, H = addMotionBlur(input_f, T, a, b)  
output_f = wienerFiltering(input_f, H, K)
```

變數 (命名不限定) :

MATLAB

input_f & output_f: 2-D image in frequency domain (centered), type single

H: motion blur degradation function in frequency domain, type single

T & a & b: motion blur parameters (見 Eq. 5.6-11 [第四版課本 Eq. 5-77])

K: Wiener filter parameter (見 Eq. 5.8-6 [第四版課本 Eq. 5-85])

Python

input_f & output_f: 2-D numpy array in frequency domain (centered), dtype=np.complex64

H: 2-D numpy array (motion blur degradation function), dtype=np.complex64

T & a & b: motion blur parameters (見 Eq. 5.6-11 [第四版課本 Eq. 5-77])

K: Wiener filter parameter (見 Eq. 5.8-6 [第四版課本 Eq. 5-85])

使用的圖片 :

Fig.5.26(a)

報告 :

- (1) 重複課本 Fig.5.26(a)(b) · 放上在 Fig.5.26(b) 加上 noise 結果和使用三種 K 得到的 Wiener filter 結果 (共 6 張圖)
- (2) 比較討論不同 K 得到的視覺效果和 PSNR
- (3) 任何想比較討論的內容或圖片 · 或者是實作心得